

SPRINT 3 – BRIEF 3

AffiliateLink: Plateforme d'affiliation

Module offer

Contexte du brief

Les advertisers d'AffiliateLink peuvent désormais créer leur compte. Il leur faut maintenant publier ce qu'ils veulent promouvoir. Vous développez le **module offer** : un advertiser actif publie des **offres** (le produit et sa page de destination), puis lance une ou plusieurs **campagnes** qui fixent la période, le budget et le modèle de rémunération des publishers (CPC, CPL ou CPA).

Ce module est le premier à dépendre d'un autre : il doit vérifier qu'un advertiser existe et que son compte est actif, sans jamais accéder aux tables du module profile. C'est le moment de mettre en place **Spring Modulith**, qui garantit que les modules restent indépendants et qu'ils communiquent uniquement par leurs interfaces publiques ou par des événements. Avant de développer le module offer, vous adaptez donc le **module profile** du brief précédent à cette architecture.

L'équipe veut aussi rendre l'application plus fiable et plus observable : les opérations qui modifient plusieurs données doivent être **transactionnelles**, et l'activité de l'application doit être **journalisée**.

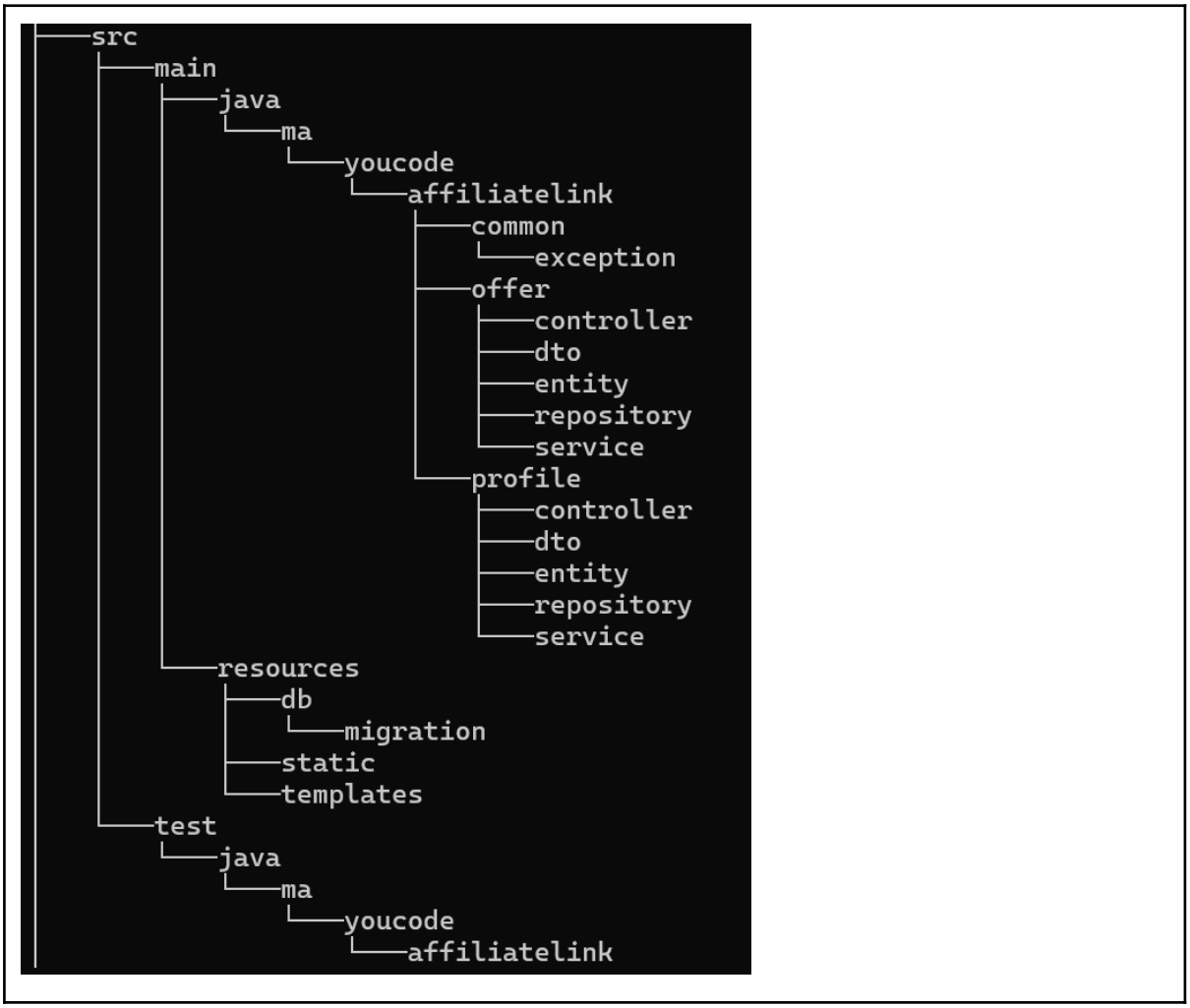
Partie 1 : adapter le module profile à Spring Modulith

Le module profile fonctionne déjà ; il s'agit de le transformer en **module Spring Modulith**, sans changer son comportement : tous les endpoints du brief précédent doivent continuer à répondre de la même façon.

Travail à faire:

1. **Ajouter Spring Modulith** au projet.
2. **Séparer le public de l'interne**.
3. **Ouvrir le module common**.
4. **Créer l'API publique du module**.
5. **Publier un événement** : quand un compte passe au statut `SUSPENDED`, le module profile publie un événement `ProfileSuspended(profileId)`.
6. **Vérifier l'architecture** : un test `ApplicationModules.of(...).verify()` passe.

Structure attendue



Partie 2 : le module offer

Spécifications fonctionnelles

User stories

- En tant qu'advertiser, je crée une offre : titre, description, URL de la page de destination, niche et mots-clés (tags).
- En tant qu'advertiser, je modifie une offre.
- En tant qu'utilisateur, je recherche des offres en combinant des critères optionnels (niche, advertiser, tag, mot-clé dans le titre), avec des résultats paginés et triés.
- En tant qu'utilisateur, je consulte une offre avec ses campagnes.
- En tant qu'advertiser, je crée une campagne pour une de mes offres : nom, dates de début et de fin, budget, modèle de rémunération et montant.
- En tant qu'advertiser, j'active, je mets en pause, je réactive ou je termine une campagne.

- En tant qu'utilisateur, je consulte les campagnes actives.

Règles de gestion

- Seul un advertiser **existant et actif** peut créer une offre ou une campagne.
- Les tags sont enregistrés en minuscules et ne sont jamais dupliqués.
- La date de fin d'une campagne est postérieure à sa date de début ; le budget est strictement positif.
- Modèles de rémunération et montant attendu :

Modèle	Signification	Montant
CPC	Paiement par clic	Montant fixe en euros, > 0
CPL	Paiement par inscription (lead)	Montant fixe en euros, > 0
CPA_FIXED	Paiement par vente, montant fixe	Montant fixe en euros, > 0
CPA_PERCENT	Paiement par vente, pourcentage	Pourcentage entre 0 et 100

- Une campagne est créée en DRAFT. Seules ces transitions sont autorisées :

Statut actuel	Transitions autorisées
DRAFT	ACTIVE
ACTIVE	PAUSED, ENDED
PAUSED	ACTIVE, ENDED
ENDED	aucune

- Une campagne dont la date de fin est dépassée ne peut pas être activée.
- **Une seule campagne active par offre** : activer une campagne met automatiquement en pause celle qui était active sur la même offre.
- Quand un advertiser est **suspendu**, toutes ses campagnes actives passent en PAUSED.

Spécifications techniques

Entités et relations

Entité	Attributs	Relations
Offre	id, advertiserId, title, description, landingUrl, niche, createdAt	@OneToMany vers Campaign (bidirectionnelle) ; @ManyToMany vers Tag
Campaign	id, name, startDate, endDate, budget, payoutModel, payoutValue, status	@ManyToOne vers Offre (côté propriétaire)

Entité	Attributs	Relations
Tag	id, name (unique)	@ManyToMany(mappedBy) vers Offre

- advertiserId est un **simple identifiant** : aucune relation JPA vers une entité du module profile.
- Montants en BigDecimal, dates en LocalDate, enums stockés en texte.

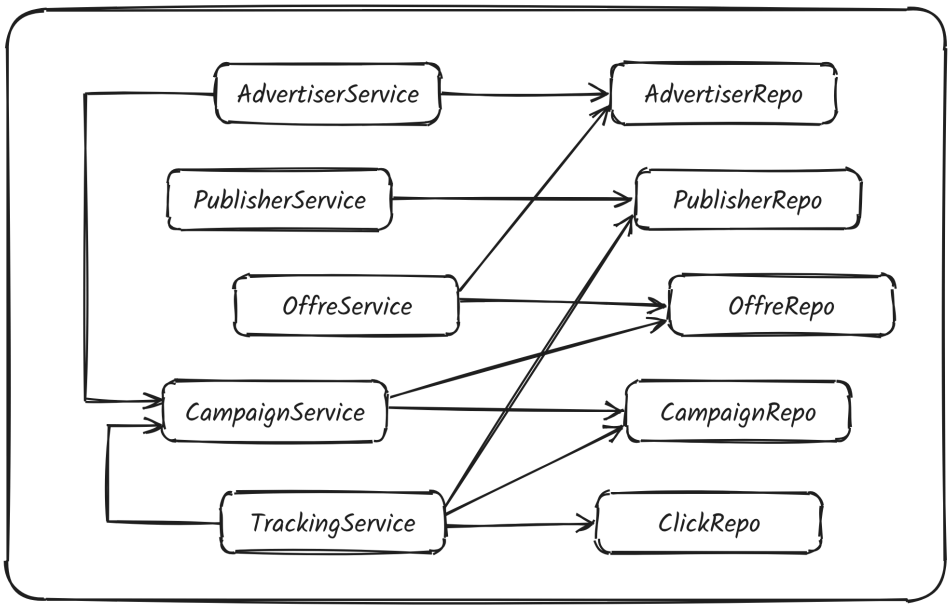
Endpoints

Méthode	URL	Réponse
POST	/api/offres	201 / 400 / 404 / 409
GET	/api/offres?niche=&advertiserId=&tag=&keyword=&page=&size=&sort=	200/400
GET	/api/offres/{id}	200 / 404
PUT	/api/offres/{id}	200 / 400 / 404
POST	/api/offres/{id}/campaigns	201 / 400 / 404 / 409
GET	/api/campaigns?status=ACTIVE	200
PATCH	/api/campaigns/{id}/status	204 / 404 / 409

package-by-layer

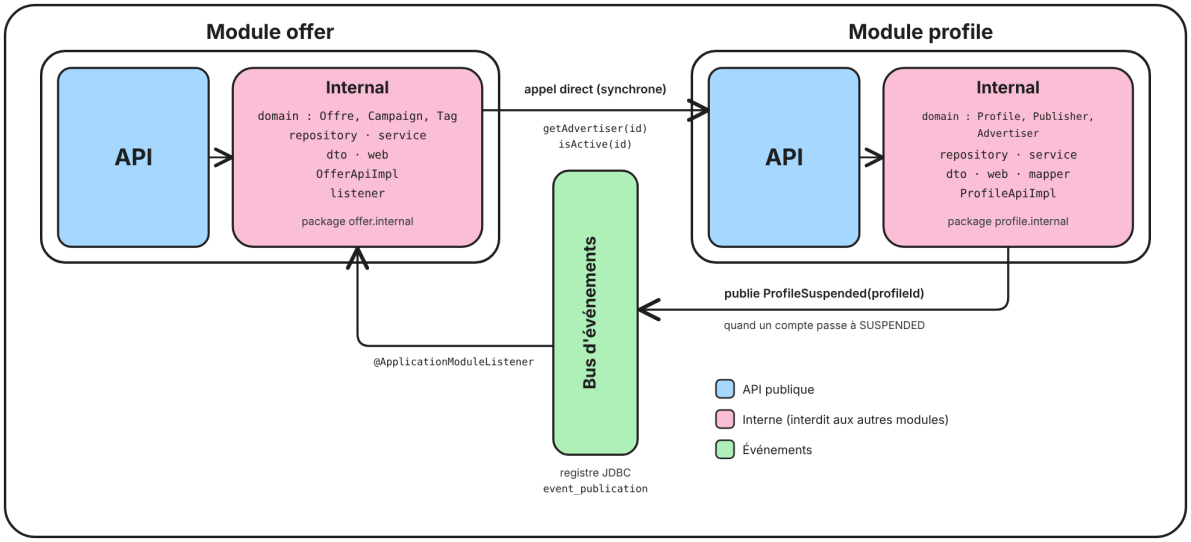
Problèmes de l'organisation par couches (package-by-layer), vos quatre points :

- la structure n'exprime pas le métier ;
- toutes les classes sont publiques et accessibles de partout ;
- cela mène à du code spaghetti ;
- une modification peut casser d'autres fonctionnalités.



Spring Modulith (package-by-feature)

Le schéma ci-dessous résume les échanges entre les deux modules : le module offer **appelle** l'API publique du module profile quand il a besoin d'une réponse, et le module profile **publie** un événement que le module offer écoute.



- Le module offer utilise le module profile **uniquement** à travers ProfileApi : aucune classe de profile n'est utilisée.
- Le module offer écoute l'événement ProfileSuspended avec @ApplicationModuleListener pour mettre en pause les campagnes de l'advertiser suspendu et aussi de notifier les publishers d'arrêter de partager les liens sur leur réseaux sociaux (envoyer des emails).
- Le **registre d'événements** est activé (spring-modulith-starter-jpa), pour qu'aucun événement ne soit perdu.

- Le module offer expose sa propre **API publique** OfferApi (getCampaign), qui servira au futur module affiliate.
- Le test verify() passe toujours avec les deux modules : aucun accès interne, aucune dépendance circulaire.

Gestion des transactions

- @Transactional sur les services ; readOnly = true pour les lectures.
- L'activation d'une campagne (mise en pause de l'ancienne campagne active, puis activation de la nouvelle) est **atomique** : si une étape échoue, rien n'est enregistré.
- Le listener de ProfileSuspended s'exécute **après la validation** de la transaction du module profile, dans **sa propre transaction**.

Logging

- Logs avec **SLF4J** (Logback par défaut dans Spring Boot) : un logger par classe..
- Niveaux utilisés: INFO pour les actions métier, WARN pour les situations anormales (transition refusée), ERROR pour les erreurs, DEBUG pour le détail.
- Logs écrits aussi dans un fichier.
- Aucune donnée sensible dans les logs.

Exigences

- Architecture en couches : contrôleur, service, repository ; injection de dépendances par constructeur.
- DTO sous forme de records en entrée et en sortie, convertis depuis et vers les entités par des mappers MapStruct (**@Mapper(componentModel = "spring")**, injectés dans les services) : aucune conversion écrite à la main, et les entités ne sont jamais exposées par l'API.
- Validation des entrées avec Bean Validation ; les règles métier (transitions de statut) sont portées par les entités.
- Erreurs renvoyées via le @RestControllerAdvice du module common : 400, 404 et 409.